

Program Overview

1. Objective

The ProLEAP Academy Live Projects program is designed to train learners to operate like project engineers rather than tutorial followers.

Each team must demonstrate four capabilities:

1. **Understand the problem** before choosing tools.
2. **Design a solution** with explicit trade-offs and measurable acceptance criteria.
3. **Build and operate the solution** using repeatable engineering practices.
4. **Explain and defend the solution** through documentation, evidence and a live demo.

2. Expected learning outcomes

By the end of a project, a learner should be able to:

- translate an ambiguous business request into testable requirements;
- identify assumptions, dependencies, risks and open questions;
- produce a Solution Intent before implementation;
- use source control and pull-request-based collaboration;
- build repeatable environments and deployment workflows;
- test functional and non-functional requirements;
- implement logging, metrics, health checks and actionable alerts;
- protect secrets and sensitive data;
- troubleshoot failures using evidence instead of guesswork;
- write an operational runbook;
- demonstrate rollback/recovery;
- present the solution to a technical and non-technical audience.

3. Project lifecycle

Phase	Primary output	Exit condition
Discovery	SCQ + stakeholder/problem notes	Problem and desired outcome are measurable
Requirements	BR/FR/NFR list + traceability matrix	Requirements are testable and prioritized
Solution Intent	Proposed architecture + trade-offs + estimate	Design review passed
Planning	Backlog, milestones, risks	Team can execute in small increments
Build	Working vertical slices	Core path works end to end
Verification	Tests + security checks + performance evidence	Acceptance criteria have evidence
Operational readiness	Dashboards, alerts, runbooks, rollback	Team can operate and recover the system
Acceptance	UAT/demo	Stakeholder can verify expected outcome
Retrospective	Lessons + improvement actions	Major decisions and failures are documented

4. Roles

A small team may combine roles, but responsibilities must remain visible.

Product Owner / Business Owner

- owns expected outcome and priority;
- validates requirements and acceptance criteria;

- makes scope decisions.

Technical Lead / Architect

- owns Solution Intent and architecture coherence;
- records major trade-offs and ADRs;
- protects non-functional requirements from being ignored.

Developer / Automation Engineer

- implements business logic, tests and automation;
- participates in code review and troubleshooting.

DevOps / Platform Engineer

- owns build/deploy automation, infrastructure, observability and operational readiness.

QA / Test Owner

- owns test strategy, test evidence and requirement coverage.

Security Champion

- checks identity, secrets, dependency risks, data handling and least privilege.

For a two-person team, combine roles deliberately and document who owns each responsibility.

5. Minimum project artifacts

Every project repository should contain at least:

```
project/
|-- README.md
|-- docs/
|   |-- SCQ.md
|   |-- SOLUTION_INTENT.md
|   |-- ARCHITECTURE.md
|   |-- REQUIREMENTS_TRACEABILITY.csv
|   |-- TEST_PLAN.md
|   |-- RUNBOOK.md
|   |-- RISK_REGISTER.csv
|   `-- ADR/
|-- src/ or app/
|-- tests/
|-- infrastructure/           # if applicable
|-- .github/workflows/ or ci/ # CI/CD
|-- .env.example
|-- VERSION_MATRIX.md
`-- CHANGELOG.md
```

6. Delivery rules

- No implementation begins before a minimally reviewable Solution Intent exists.
- A requirement without an acceptance method is incomplete.
- A feature without a test/evidence path is not considered complete.
- A deployment that cannot be reproduced from documentation or automation is not considered complete.
- A critical alert without a runbook is incomplete.
- A secret in source control is an automatic security failure until rotated and remediated.
- A demo must include at least one controlled failure and recovery path, not only the happy path.

7. Scope control

Teams should use MoSCoW priority:

- **Must:** necessary for project acceptance.
- **Should:** high value, but can be deferred if time is constrained.
- **Could:** enhancement after Must/Should are stable.
- **Won't now:** deliberately out of scope for the cohort.

Do not expand scope just to add more tools. A smaller system with strong tests, observability, security and recovery is better than a broad but fragile demo.

8. Evidence policy

Acceptable evidence includes:

- automated test output;
- CI/CD run links/screenshots;
- API request/response examples;
- database query results with sanitized data;
- dashboards and alert screenshots;
- infrastructure plan/apply output;
- logs from controlled failure scenarios;
- recorded demo clips;
- runbook execution evidence.

Screenshots alone are not enough if the result can be automatically tested.

9. Relationship to the ProLEAP DevOps learning path

The live projects are intended to reinforce the same engineering chain used in the broader learning plan: Linux -> Git -> automation -> CI/CD -> containers -> IaC -> cloud -> platform operations -> observability -> security. The projects should therefore generate portfolio artifacts, not isolated exercises.